

BÁO CÁO

Họ & Tên: Vũ Quốc Hùng	MSSV: 24207015
Lớp: 24DTV-DKD1	Ca: 15-17h

Bài 02 CÁC THUẬT TOÁN TÌM KIẾM

Bài tập bắt buộc:

3. Hãy viết hàm đếm số đường chạy của mảng một chiều a có n phần tử.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#define N 100

void nhapmang(int *pa, int *n) {
    printf("nhap so phan tu cua mang: ");
    scanf("%d", n);
    for (int i = 0; i < *n; i++) {
        printf("nhap a[%d]: ", i);
        scanf("%d", (pa + i));
    }
}

void xuatmang(int *pa, int len) {
    for (int i = 0; i < len; i++)
        printf("%d ", *(pa + i));
    printf("\n");
}

void demsoduongchay(int *pa, int n) {
    if (n < 2) {
        printf("so duong chay: 0\n");
        return;
    }
    int runs = 0;
    int cur_length = 1;
    for (int i = 1; i < n; i++) {
        if (*(pa + i) > *(pa + i - 1)) {
            cur_length++;
        } else {
            if (cur_length >= 2) {
                runs++;
            }
            cur_length = 1;
        }
    }
    if (cur_length >= 2) {
        runs++;
    }
    printf("so duong chay: %d\n", runs);
}

int main() {
    int a[N];
    int n = 0;
    nhapmang(a, &n);
    printf("mang vua nhap: ");
    xuatmang(a, n);
    demsoduongchay(a, n);
}
```

```

    return 0;
}

```

Giải thích cách hoạt động của code:

1. Hàm **nhapmang** và **xuatmang**: Đã trình bày ở tuần 1, em tận dụng lại dựa trên cấu trúc bài code thầy Thuận dùng để giảng các loại sort trong buổi 1, 2 của học phần lý thuyết Cấu trúc dữ liệu giải thuật (sử dụng con trỏ để thu thập dữ liệu đầu vào và in dữ liệu ra màn hình).

2. Hàm **demsoduongchay**: Theo lý thuyết thầy giảng trong lớp đường chạy là mảng tăng từ 2 phần tử trở lên => mảng có dưới 2 phần tử không tạo thành 1 đường chạy. Ý tưởng của thuật toán là duyệt qua mảng để duy trì và kéo dài một 'đường chạy' tạm thời. Nếu phần tử sau lớn hơn phần tử trước, chiều dài đường chạy (**cur_length**) tăng lên. Ngược lại, khi chuỗi tăng bị ngắt, ta tiến hành kiểm tra xem đường chạy vừa qua có đủ điều kiện (độ dài ≥ 2) hay không để ghi nhận vào biến đếm (**runs**), sau đó thiết lập lại trạng thái để bắt đầu một đường chạy mới. (các bước cụ thể em minh họa ở bảng dưới)

3. Hàm **main**: Gọi lần lượt các hàm để nhận n phần tử từ bàn phím, in mảng và in ra tổng số đường chạy.

Ví dụ minh họa: Dãy được cho ở lớp: 8 5 5 2 3 3 4

Lượt quét	Phần tử xét ($a[i]$)	Số sánh với trước ($a[i-1]$)	Trạng thái chuỗi	Chiều dài hiện tại (current_length)	Tổng đường chạy (runs)	Diễn giải hành động
Khởi tạo	-	-	-	1	0	Bắt đầu xét mốc từ phần tử đầu tiên
i = 1	$a[1] = 5$	$a[0] = 8$	$5 < 8$ (Giảm)	$1 \Rightarrow 1$	0	Chuỗi ngắt. Bỏ qua dãy [8] do độ dài < 2 . Reset chiều dài.
i = 2	$a[2] = 5$	$a[1] = 5$	$5 = 5$ (Bằng)	$1 \Rightarrow 1$	0	Chuỗi ngắt. Bỏ qua dãy [5] do độ dài < 2 . Reset chiều dài.
i = 3	$a[3] = 2$	$a[2] = 5$	$2 < 5$ (Giảm)	$1 \Rightarrow 1$	0	Chuỗi ngắt. Bỏ qua dãy [5] do độ dài < 2 . Reset chiều dài.
i = 4	$a[4] = 3$	$a[3] = 2$	$3 > 2$ (Tăng)	$1 \Rightarrow 2$	0	Thỏa điều kiện tăng. Ghi nhận tiến trình gom dãy [2, 3].
i = 5	$a[5] = 3$	$a[4] = 3$	$3 = 3$ (Bằng)	$2 \Rightarrow 1$	$0 \Rightarrow 1$	Chuỗi ngắt. Dãy [2, 3] đã gom hợp lệ (độ dài ≥ 2) -> Cộng 1 đường chạy.

						Reset chiều dài.
i = 6	a[6] = 4	a[5] = 3	4 > 3 (Tăng)	1 => 2	1	Thỏa điều kiện tăng dần. Gom được dãy [3, 4]. Tăng biến chiều dài lên 2.
Kết thúc	-	-	-	2	1 => 2	Hết vòng lặp duyệt mảng. Chốt dãy đang gom dở [3, 4], đủ điều kiện chiều dài >= 2 => Ghi nhận thêm 1 đường chạy.

Kết quả:

```

PS D:\daihoc\nam2\HK3\THDSA\Tuan2> .\btbb3
nhap so phan tu cua mang: 7
nhap a[0]: 8
nhap a[1]: 5
nhap a[2]: 5
nhap a[3]: 2
nhap a[4]: 3
nhap a[5]: 3
nhap a[6]: 4
mang vua nhap: 8 5 5 2 3 3 4
so duong chay: 2

```

Bài tập nâng cao: Định nghĩa struct Sinh viên:

+ Ngày sinh: ngày, tháng, năm là số nguyên không dấu,

+ MSSV: ký tự 10 chữ số.

+ Họ và tên: 20 kí tự

+ Điểm trung bình: float

1. Viết hàm đọc mảng sinh viên từ file "input.txt".

2. Tìm sinh viên có ngày sinh nhỏ nhất và hiển thị lên màn hình.

3. Tìm sinh viên có điểm trung bình theo yêu cầu (nhập từ bàn phím).

4. Tìm sinh viên có mssv theo yêu cầu bằng thuật toán tìm kiếm nhị phân (nhập từ bàn phím).

Code:

```
#include <stdio.h>
#include <stdlib.h>
#define N 100

typedef struct {
    char ngay[3];
    char thang[3];
    char nam[5];
} NgaySinh;
typedef struct {
    NgaySinh ngaysinh;
    char mssv[11];
    char hoten[21];
    float dtb;
} SinhVien;
int docfile(char *tenFile, SinhVien *ds_sv) {
    FILE *fi = fopen(tenFile, "rt");
    if (fi == NULL) {
        printf("khong the mo file %s\n", tenFile);
        return 0;
    }
    int count = 0;
    while (count < N) {
        if (fscanf(fi, " %10[^\n] %20[^\n] %2[0-9]/%2[0-9]/%4[0-9] %f",
            ds_sv[count].mssv, ds_sv[count].hoten,
            ds_sv[count].ngaysinh.ngay, ds_sv[count].ngaysinh.thang,
            ds_sv[count].ngaysinh.nam, &ds_sv[count].dtb) != 6)
            break;
        count++;
    }
    fclose(fi);
    return count;
}
void xuatsv(SinhVien sv) {
    printf("MSSV: %-10s - Ho ten: %-30s - Ngay sinh: %s/%s/%s - DTB: %.2f\n",
        sv.mssv, sv.hoten, sv.ngaysinh.ngay, sv.ngaysinh.thang,
```

```

        sv.ngaysinh.nam, sv.dtb);
    }
    void xuatfile(SinhVien *ds_sv, int n) {
        for (int i = 0; i < n; i++)
            xuatsv(ds_sv[i]);
    }
    int sosanhngaysinh(NgaySinh ns1, NgaySinh ns2) {
        int n1 = atoi(ns1.nam), n2 = atoi(ns2.nam);
        int t1 = atoi(ns1.thang), t2 = atoi(ns2.thang);
        int d1 = atoi(ns1.ngay), d2 = atoi(ns2.ngay);
        if (n1 != n2)
            return (n1 < n2) ? -1 : 1;
        if (t1 != t2)
            return (t1 < t2) ? -1 : 1;
        if (d1 != d2)
            return (d1 < d2) ? -1 : 1;
        return 0;
    }
    void timsvngaysinhnhohat(SinhVien *ds_sv, int n) {
        if (n == 0)
            return;
        int idx_min = 0;
        for (int i = 1; i < n; i++) {
            if (sosanhngaysinh(ds_sv[i].ngaysinh, ds_sv[idx_min].ngaysinh) < 0)
                idx_min = i;
        }
        printf("sinh vien co ngay sinh nho nhat la:\n");
        xuatsv(ds_sv[idx_min]);
    }
    void timsvdtb(SinhVien *ds_sv, int n) {
        float diem_yc;
        printf("nhap diem trung binh can tim: ");
        scanf("%f", &diem_yc);
        int found = 0;
        for (int i = 0; i < n; i++) {
            if (ds_sv[i].dtb == diem_yc) {
                xuatsv(ds_sv[i]);
                found++;
            }
        }
        if (found == 0)
            printf("khong tim thay sinh vien nao co DTB = %.2f\n", diem_yc);
    }
    void sapxeptheomssv(SinhVien *ds_sv, int n) {
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                int k = 0;
                while (ds_sv[i].mssv[k] != '\0' && ds_sv[j].mssv[k] != '\0') {

```

```

        if (ds_sv[i].mssv[k] != ds_sv[j].mssv[k])
            break;
        k++;
    }
    if (ds_sv[i].mssv[k] > ds_sv[j].mssv[k]) {
        SinhVien temp = ds_sv[i];
        ds_sv[i] = ds_sv[j];
        ds_sv[j] = temp;
    }
}
}
}
}
void timsvmss(SinhVien *ds_sv, int n) {
    char mssv_yc[11];
    printf("nhap MSSV can tim: ");
    scanf("%10s", mssv_yc);
    sapxeptheomssv(ds_sv, n);
    int left = 0, right = n - 1, found_idx = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int k = 0, cmp = 0;
        while (ds_sv[mid].mssv[k] != '\0' && mssv_yc[k] != '\0') {
            if (ds_sv[mid].mssv[k] != mssv_yc[k]) {
                cmp = (ds_sv[mid].mssv[k] > mssv_yc[k]) ? 1 : -1;
                break;
            }
            k++;
        }
        if (cmp == 0) {
            if (ds_sv[mid].mssv[k] == '\0' && mssv_yc[k] == '\0') {
                found_idx = mid;
                break;
            } else {
                cmp = (ds_sv[mid].mssv[k] == '\0') ? -1 : 1;
            }
        }
        if (cmp < 0)
            left = mid + 1;
        else
            right = mid - 1;
    }
    if (found_idx != -1) {
        xuatsv(ds_sv[found_idx]);
    } else {
        printf("khong tim thay sinh vien co MSSV %s\n", mssv_yc);
    }
}
int main() {

```

```

SinhVien ds_sv[N];
int n = docfile("input.txt", ds_sv);
if (n == 0) {
    printf("khong co du lieu hoac file bi loi\n");
    return 1;
}
xuatfile(ds_sv, n);
timsvngaysinhnhohat(ds_sv, n);
timsvdtb(ds_sv, n);
timsvmss(ds_sv, n);
return 0;
}

```

Giải thích cách hoạt động của code:

1. Cơ chế cắt trường dữ liệu tùy ý em dùng hàm `fscanf` trong `docfile` với chuỗi định dạng được sử dụng là: " %10[^\n] %20[^\n] %2[0-9]/%2[0-9]/%4[0-9] %f".

- Ký tự trắng ở khoảng đầu để bỏ qua tất cả các ký tự khoảng trắng (gồm dấu cách, tab, xuống dòng \n) cho đến khi gặp ký tự có nghĩa nhờ vậy dữ liệu có thể nằm ở cùng 1 dòng hoặc trên 1 dòng tùy ý.
- xx[^\n]: yêu cầu đọc một chuỗi tối đa xx ký tự hoặc dừng lại ngay lập tức nếu gặp \n (em tìm ra cách này tối ưu hơn %s, nếu gặp dấu cách là ngắt nên muốn gõ tên phải dùng thêm dấu _ giữa các ký tự và xx sẽ ứng với từng yêu cầu giới hạn ký tự của đề bài 10 cho MSSV và 20 cho Họ và tên).
- %2[0-9]: bắt buộc phải đọc ký tự từ 0 đến 9. kết hợp %2[0-9]/%2[0-9]/%4[0-9] để bắt buộc dữ liệu ngày tháng theo đúng form DD/MM/YYYY.

Khi `fscanf` thu thập đủ 6 trường dữ liệu (mssv, hoten, ngay, thang, nam, dtb) thì thỏa điều kiện `!=6` để thoát khỏi vòng lặp

File `input.txt` có nội dung như sau:

```
input.txt
File Edit View

24207088
Nguyen Thai TDat
18/02/2006
9.1
24207015
Vu Quoc Hung
23/09/2006
8.6
24207014
Nguyen Minh Hoang
03/02/2006
8.6
24207032
Nguyen Tan Phat
18/06/2006
8.0
24207092
Do Hoang Dat
09/06/2006
7.9
24207030
Doan Minh Nhat
16/01/2003
7.5
24207012
Dinh Huy Hoang Hieu
11/04/2006
8.4
24207036
Nguyen Thanh Phu
28/03/2006
8.4
```

2. Hàm `sosanhngaysinh`:

Hàm này nhận vào hai tham số kiểu `NgaySinh` (chứa các chuỗi ký tự ngày, tháng, năm) và trả về một số nguyên để xác định mốc thời gian nào nhỏ hơn (tức là sinh ra trước/lớn tuổi hơn): Do ngày/tháng/năm lưu dạng chuỗi, em dùng `atoi()` trong thư viện `<stdlib.h>` để chuyển sang số nguyên rồi so sánh. (thầy kêu không được dùng hàm có sẵn trong thư viện nhưng mà chỗ này em chưa biết xử lý sao cho khéo hơn nên mong thầy thông cảm ạ)

Trình tự so sánh:

- Năm ưu tiên trước => nhỏ hơn => trả về -1. Lớn hơn => trả về 1
- Nếu năm bằng nhau => xét tháng => nhỏ hơn => trả về -1. Lớn hơn => trả về 1
- Nếu cả năm và tháng bằng nhau => xét ngày => nhỏ hơn => trả về -1. Lớn hơn => trả về 1
- Nếu trùng khớp hoàn toàn: Nếu cả năm, tháng, ngày đều bằng nhau => trả về 0, nghĩa là 2 ngày sinh hoàn toàn giống nhau.

3. Các hàm Linear Search:

- `timsvngaysinhnhohat`: Khởi tạo vị trí nhỏ nhất `idx_min = 0`. Duyệt tuần tự mảng từ `i = 1`. Lần lượt dùng hàm so sánh tự định nghĩa (`sosanhngaysinh`) để tìm ra bạn có năm/tháng/ngày nhỏ hơn, từ đó cập nhật lại `idx_min`. Số lần so sánh tối đa là `n - 1`.

Để minh họa em có sửa đổi code ở hàm `timsvngaysinhnhohat` để xem xét từng bước xét tìm ra min:

```
void timsvngaysinhnhohat(SinhVien *ds_sv, int n) {
```



```

if (n == 0)
    return;
int idx_min = 0;
for (int i = 1; i < n; i++) {
    printf("buoc %d: so sanh [%s] voi min hien tai [%s]... ", i,
ds_sv[i].hoten,
        ds_sv[idx_min].hoten);
    if (sosanhngaysinh(ds_sv[i].ngaysinh, ds_sv[idx_min].ngaysinh) < 0) {
        idx_min = i;
        printf("=> nho hon => cap nhat min moi\n");
    } else {
        printf("=> bo qua\n");
    }
}
printf("\nsinh vien co ngay sinh nho nhat:\n");
xuatsv(ds_sv[idx_min]);
}

```

Kết quả:

buoc 1: so sanh [Vu Quoc Hung] voi min hien tai [Nguyen Thai TDat]=> bo qua
 buoc 2: so sanh [Nguyen Minh Hoang] voi min hien tai [Nguyen Thai TDat]=> nho hon => cap nhat min moi
 buoc 3: so sanh [Nguyen Tan Phat] voi min hien tai [Nguyen Minh Hoang]=> bo qua
 buoc 4: so sanh [Do Hoang Dat] voi min hien tai [Nguyen Minh Hoang]=> bo qua
 buoc 5: so sanh [Doan Minh Nhat] voi min hien tai [Nguyen Minh Hoang]=> nho hon => cap nhat min moi
 buoc 6: so sanh [Dinh Huy Hoang Hieu] voi min hien tai [Doan Minh Nhat]=> bo qua
 buoc 7: so sanh [Nguyen Thanh Phu] voi min hien tai [Doan Minh Nhat]=> bo qua
 sinh vien co ngay sinh nho nhat:

MSSV: 24207030 - Ho ten: Doan Minh Nhat - Ngay sinh: 16/01/2003 - DTB: 7.50

- **timsvdtb**: Nhập ĐTB. Duyệt tuần tự từ đầu đến cuối mảng (i = 0 đến n-1). Tại mỗi bước tiến hành kiểm tra ds_sv[i].dtb == diem_yc. Nếu tìm thấy, in ra ngay lập tức và tăng biến cờ found. (đảm bảo liệt kê ra được nhiều hơn 1 người đủ điều kiện)
 Tương tự bên trên để minh họa:

```

void timsvdtb(SinhVien *ds_sv, int n) {
    float diem_yc;
    printf("nhap diem trung binh can tim: ");
    scanf("%f", &diem_yc);
    int found = 0;
    for (int i = 0; i < n; i++) {
        printf("buoc %d: kiem tra index %d (%s) co DTB %.2f ", i + 1, i,
            ds_sv[i].hoten, ds_sv[i].dtb);
        if (ds_sv[i].dtb == diem_yc) {
            printf("=> khop\n");
            xuatsv(ds_sv[i]);
            found++;
        } else {
            printf("=> khong khop\n");
        }
    }
    if (found == 0)
        printf("khong tim thay sinh vien nao co DTB = %.2f\n", diem_yc);
}

```

Kết quả:

```
nhap diem trung binh can tim: 8.4
buoc 1:kiem tra index 0 (Nguyen Thai TDat) co DTB 9.10 => khong khop
buoc 2:kiem tra index 1 (Vu Quoc Hung) co DTB 8.60 => khong khop
buoc 3:kiem tra index 2 (Nguyen Minh Hoang) co DTB 8.60 => khong khop
buoc 4:kiem tra index 3 (Nguyen Tan Phat) co DTB 8.00 => khong khop
buoc 5:kiem tra index 4 (Do Hoang Dat) co DTB 7.90 => khong khop
buoc 6:kiem tra index 5 (Doan Minh Nhat) co DTB 7.50 => khong khop
buoc 7:kiem tra index 6 (Dinh Huy Hoang Hieu) co DTB 8.40 => khop
MSSV: 24207012 - Ho ten: Dinh Huy Hoang Hieu - Ngay sinh: 11/04/2006 - DTB: 8.40
buoc 8:kiem tra index 7 (Nguyen Thanh Phu) co DTB 8.40 => khop
MSSV: 24207036 - Ho ten: Nguyen Thanh Phu - Ngay sinh: 28/03/2006 - DTB: 8.40
```

4. Hàm Binary Search:

Tim mssv bằng hàm nhị phân `timsvmss` nhưng trước đó mảng phải được sắp xếp nên em dùng thêm 1 hàm `sapxeptheomssv` (dùng Selection Sort) để lấy dữ liệu ở trạng thái tăng dần.

Quá trình tìm kiếm:

- Đặt 2 mốc `left = 0` và `right = n - 1`
- Trong khi không gian tìm kiếm chưa rỗng (`left <= right`), lấy phần tử ở giữa `mid = left + (right - left) / 2`
- So sánh từng ký tự của MSSV tại `mid` và MSSV cần tìm. Quá trình so sánh và trả về kết quả `< 0, > 0, == 0`
- Nếu khớp: ghi nhận `found_idx = mid` và ngắt vòng lặp
- Nếu MSSV ở `mid` lớn hơn MSSV yêu cầu => kết quả (nếu có) phải nằm ở nửa bên trái => thu hẹp không gian bằng cách gán `right = mid - 1`
- Nếu MSSV ở `mid` nhỏ hơn => thu hẹp về nửa phải bằng `left = mid + 1`

Tương tự với các hàm Linear Search, em cũng minh họa bằng cách thêm hàm để xem từng bước:

```
void timsvmss(SinhVien *ds_sv, int n) {
    char mssv_yc[11];
    printf("nhap MSSV can tim: ");
    scanf("%10s", mssv_yc);
    sapxeptheomssv(ds_sv, n);
    int left = 0, right = n - 1, found_idx = -1;
    int buoc = 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        printf("buoc %d: left = %d, right = %d -> mid = %d.\n", buoc++, left,
            right,
                mid);
        printf("          dang xet MSSV tai mid: [%s]... ", ds_sv[mid].mssv);

        int k = 0, cmp = 0;
        while (ds_sv[mid].mssv[k] != '\0' && mssv_yc[k] != '\0') {
            if (ds_sv[mid].mssv[k] != mssv_yc[k]) {
                cmp = (ds_sv[mid].mssv[k] > mssv_yc[k]) ? 1 : -1;
                break;
            }
            k++;
        }
    }
}
```

```

    }
    if (cmp == 0) {
        if (ds_sv[mid].mssv[k] == '\0' && mssv_yc[k] == '\0') {
            found_idx = mid;
            printf("=> khop\n");
            break;
        } else {
            cmp = (ds_sv[mid].mssv[k] == '\0') ? -1 : 1;
        }
    }
    if (cmp < 0) {
        printf("=> nho hon => thu hep ve nua phai (left = %d).\n", mid + 1);
        left = mid + 1;
    } else {
        printf("=> lon hon => thu hep ve nua trai (right = %d).\n", mid - 1);
        right = mid - 1;
    }
}
if (found_idx != -1) {
    xuatsv(ds_sv[found_idx]);
} else {
    printf("khong tim thay sinh vien co MSSV %s\n", mssv_yc);
}
}

```

Kết quả ví dụ 1:

nhap MSSV can tim: 24207012
 buoc 1: left = 0, right = 7 -> mid = 3.
 dang xet MSSV tai mid: [24207030]... => lon hon => thu hep ve nua trai (right = 2).
 buoc 2: left = 0, right = 2 -> mid = 1.
 dang xet MSSV tai mid: [24207014]... => lon hon => thu hep ve nua trai (right = 0).
 buoc 3: left = 0, right = 0 -> mid = 0.
 dang xet MSSV tai mid: [24207012]... => khop
 MSSV: 24207012 - Ho ten: Dinh Huy Hoang Hieu - Ngay sinh: 11/04/2006 - DTB: 8.40

Kết quả ví dụ 2:

nhap MSSV can tim: 24207092
 buoc 1: left = 0, right = 7 -> mid = 3.
 dang xet MSSV tai mid: [24207030] => nho hon => thu hep ve nua phai (left = 4)
 buoc 2: left = 4, right = 7 -> mid = 5.
 dang xet MSSV tai mid: [24207036] => nho hon => thu hep ve nua phai (left = 6)
 buoc 3: left = 6, right = 7 -> mid = 6.
 dang xet MSSV tai mid: [24207088] => nho hon => thu hep ve nua phai (left = 7)
 buoc 4: left = 7, right = 7 -> mid = 7.
 dang xet MSSV tai mid: [24207092] => khop
 MSSV: 24207092 - Ho ten: Do Hoang Dat - Ngay sinh: 09/06/2006 - DTB: 7.90

Kết quả:

Em khảo sát 1 vài trường với file input.txt khác nhau:

TH1: Không có file input

[Running] cd "d:\daihoc\nam2\HK3\THDSA\Tuan2\" && gcc btnc.c -o btnc &&

khong the mo file input.txt

khong co du lieu hoac file bi loi

TH2: File input không đúng điều kiện đề bài (MSSV: ký tự 10 chữ số & Họ và tên: 20 ký tự)

Vdu: Bạn tên là Nguyen Thai Thanh Dat (tính các ký tự chữ và dấu cách tổng là 21 vượt quá giới hạn)

```
[Running] cd "d:\daihoc\nam2\HK3\THDSA\Tuan2\" && gcc btnc.c -o b
khong co du lieu hoac file bi loi
```

TH3: Dữ liệu trong file đúng chuẩn điều kiện với các thông tin cần tìm là ĐTB = 8.4 và MSSV là 24207014 (có tồn tại trong file input)

```
PS D:\daihoc\nam2\HK3\THDSA\Tuan2> .\btnc
MSSV: 24207088 - Ho ten: Nguyen Thai TDat - Ngay sinh: 18/02/2006 - DTB: 9.10
MSSV: 24207015 - Ho ten: Vu Quoc Hung - Ngay sinh: 23/09/2006 - DTB: 8.60
MSSV: 24207014 - Ho ten: Nguyen Minh Hoang - Ngay sinh: 03/02/2006 - DTB: 8.60
MSSV: 24207032 - Ho ten: Nguyen Tan Phat - Ngay sinh: 18/06/2006 - DTB: 8.00
MSSV: 24207092 - Ho ten: Do Hoang Dat - Ngay sinh: 09/06/2006 - DTB: 7.90
MSSV: 24207030 - Ho ten: Doan Minh Nhat - Ngay sinh: 16/01/2003 - DTB: 7.50
MSSV: 24207012 - Ho ten: Dinh Huy Hoang Hieu - Ngay sinh: 11/04/2006 - DTB: 8.40
MSSV: 24207036 - Ho ten: Nguyen Thanh Phu - Ngay sinh: 28/03/2006 - DTB: 8.40
sinh vien co ngay sinh nho nhat la:
MSSV: 24207030 - Ho ten: Doan Minh Nhat - Ngay sinh: 16/01/2003 - DTB: 7.50
nhap diem trung binh can tim: 8.4
MSSV: 24207012 - Ho ten: Dinh Huy Hoang Hieu - Ngay sinh: 11/04/2006 - DTB: 8.40
MSSV: 24207036 - Ho ten: Nguyen Thanh Phu - Ngay sinh: 28/03/2006 - DTB: 8.40
nhap MSSV can tim: 24207014
MSSV: 24207014 - Ho ten: Nguyen Minh Hoang - Ngay sinh: 03/02/2006 - DTB: 8.60
```

TH4: Các thông tin cần tìm là ĐTB = 8.1 và MSSV là 24207019 (không tồn tại trong file input)

```
nhap diem trung binh can tim: 8.1
khong tim thay sinh vien nao co DTB = 8.10
nhap MSSV can tim: 24207019
khong tim thay sinh vien co MSSV 24207019
PS D:\daihoc\nam2\HK3\THDSA\Tuan2> █
```